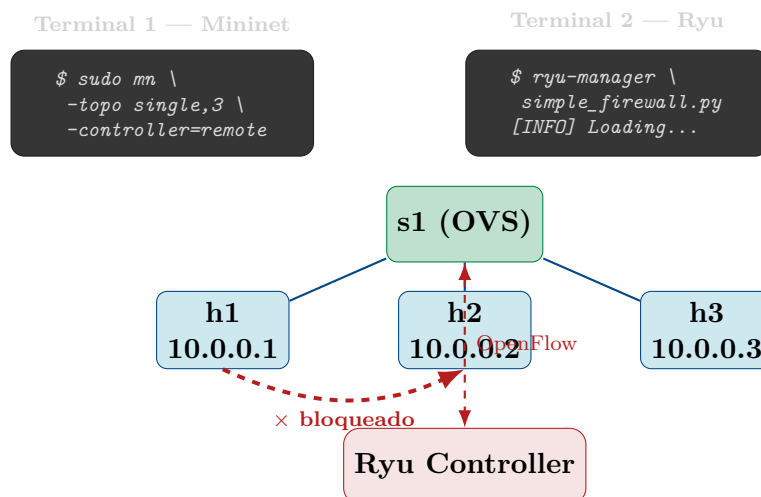

GUÍA DE LABORATORIO

Laboratorio 2

Introducción práctica a SDN con Mininet y Ryu



Tema: Redes Definidas por Software (SDN)
Herramientas: Mininet · Ryu · Open vSwitch (OVS) · Ubuntu
Duración: 2 horas (grupos) / 1.5 h + 0.5 h discusión
Nivel: Pregrado — Ingeniería de Sistemas

Índice

Objetivos del laboratorio	2
0. Requisitos previos	3
Paso 0: Requisitos previos	3
1. Instalación de Mininet y Ryu	4
1.1. Actualizar paquetes del sistema	4
1.2. Instalar Mininet	4
1.3. Instalar Ryu (controlador SDN)	4
2. Primera topología SDN con Mininet	6
2.1. Crear una topología simple (1 switch, 3 hosts)	6
2.2. Explorar la topología desde la consola de Mininet	6
3. Ejecutar un controlador SDN básico (Ryu)	8
3.1. Ejecutar el controlador <code>simple_switch_13</code>	8
3.2. Verificar conectividad con el controlador activo	8
4. Inspeccionar flujos en el switch (Open vSwitch)	10
4.1. Ver flujos instalados en <code>s1</code>	10
5. Modificar el comportamiento de la red: firewall simple	11
5.1. Detener Mininet y Ryu	11
5.2. Crear el directorio de trabajo y el script	11
5.3. Código del controlador <code>simple_firewall.py</code>	11
6. Probar el controlador firewall en Mininet	15
6.1. Levantar Mininet (Terminal 1)	15
6.2. Ejecutar el firewall propio (Terminal 2)	15
6.3. Verificar IPs de los hosts	15
6.4. Prueba 1: <code>pingall</code>	15
6.5. Pruebas específicas	16
7. Variaciones propuestas y entregable	17
7.1. Variación 1: cambiar la regla de bloqueo	17
7.2. Variación 2: bloquear todo el tráfico hacia un host	17
7.3. Comparación Lab 1 vs. Lab 2	17
7.4. Entregable: reflexión escrita	18
Resumen del laboratorio	19
Bibliografía	20

Objetivos del laboratorio

Objetivos del laboratorio

Al finalizar el laboratorio el estudiante será capaz de:

1. Desplegar una topología virtual SDN usando **Mininet**.
2. Ejecutar un controlador SDN (**Ryu**) y conectar switches virtuales.
3. Observar cómo el controlador **programa flujos** en los switches.
4. **Modificar** un controlador simple para cambiar el comportamiento de la red (bloquear cierto tráfico).

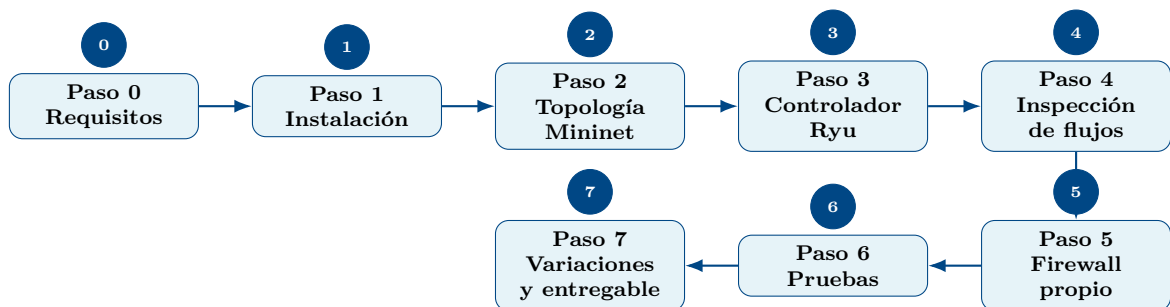
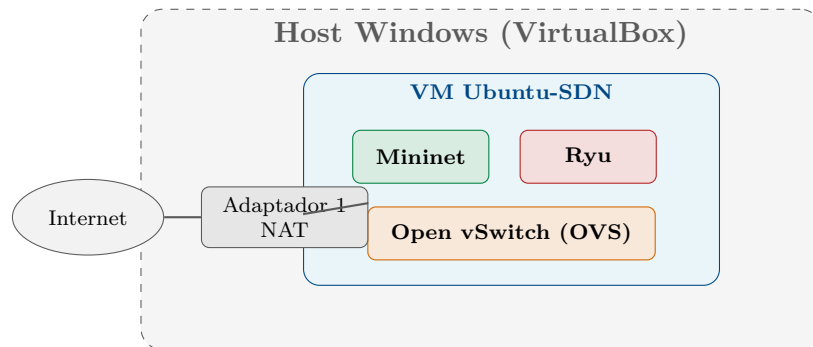


Figura 1: Flujo de trabajo del laboratorio.

Paso 0: Requisitos previos

- PC con **Windows** + **VirtualBox** instalado.
- VM **Ubuntu-SDN** con:
 - Acceso a Internet (*Adaptador 1: NAT*).
 - Usuario con privilegios **sudo** (ej. **alumno**).



Terminal 1 Terminal 2

Figura 2: Configuración de la VM en VirtualBox para el laboratorio.

Consejo

Configura la VM con **Adaptador 1: NAT** antes de arrancar. Esto permite instalar paquetes desde Internet sin configuración adicional de red.

Paso 1: Instalación de Mininet y Ryu

1.1 Actualizar paquetes del sistema

Abre una terminal en **Ubuntu-SDN** y ejecuta:

Listing 1: Actualizar repositorios y paquetes.

```
sudo apt update
sudo apt upgrade -y
```

1.2 Instalar Mininet

Listing 2: Instalación de Mininet desde paquete Debian.

```
sudo apt install -y mininet
```

Verifica la instalación:

Listing 3: Verificar versión de Mininet.

```
mn --version
```

Salida esperada

```
2.3.0
```

1.3 Instalar Ryu (controlador SDN)

Instala primero **pip** para Python 3:

Listing 4: Instalar pip3.

```
sudo apt install -y python3-pip
```

Luego instala **Ryu**:

Listing 5: Instalar Ryu mediante pip3.

```
sudo pip3 install ryu
```

Verifica que **Ryu** esté disponible:

Listing 6: Verificar versión de Ryu.

```
ryu-manager --version
```

Salida esperada

```
ryu-manager 4.34
```

Concepto clave

Mininet emula una red completa (hosts, switches, enlaces) *dentro de una sola VM*.

Ryu es el controlador SDN que programará los switches virtuales mediante OpenFlow.

Paso 2: Primera topología SDN con Mininet

2.1 Crear una topología simple (1 switch, 3 hosts)

Listing 7: Levantar topología `single,3` con controlador remoto.

```
sudo mn --topo single,3 --controller=remote --switch ovsk
```

Argumento	Descripción
<code>-topo single,3</code>	1 switch + 3 hosts conectados.
<code>-controller=remote</code>	El switch busca un controlador SDN externo.
<code>-switch ovsk</code>	Open vSwitch (compatible con OpenFlow).

Cuadro 1: Parámetros del comando `mn`.

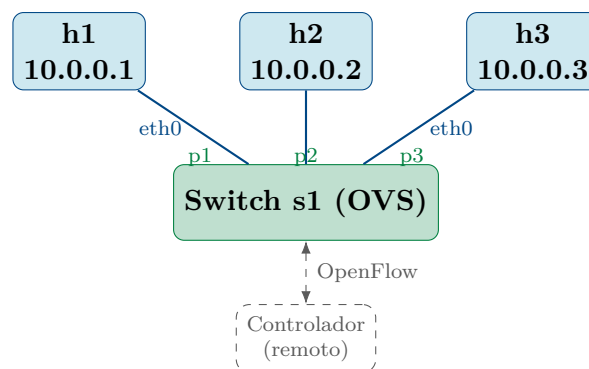


Figura 3: Topología `single,3` creada por Mininet.

2.2 Explorar la topología desde la consola de Mininet

Una vez que aparezca el prompt `mininet>`, ejecuta:

Listing 8: Comandos de exploración en Mininet.

```
mininet> nodes
mininet> net
```

- `nodes` — muestra los nodos: `h1`, `h2`, `h3`, `s1`.
- `net` — muestra las conexiones entre interfaces.

Prueba un `pingall` *antes* de levantar el controlador:

```
mininet> pingall
```

Atención

Sin controlador activo los paquetes no tienen ruta y los pings **fallarán o se comportarán de forma impredecible**. Esto es normal y es parte del aprendizaje.

Paso 3: Ejecutar un controlador SDN básico (Ryu)

Atención

Necesitas **dos terminales** abiertas en la misma VM. La Terminal 1 ejecuta Mininet; la Terminal 2 ejecuta Ryu.

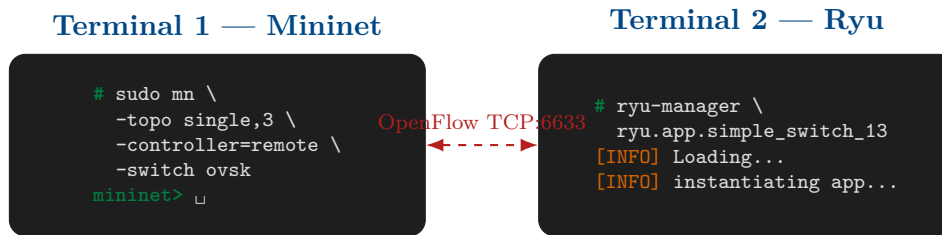


Figura 4: Las dos terminales necesarias para el laboratorio.

3.1 Ejecutar el controlador `simple_switch_13`

En la **Terminal 2**:

Listing 9: Lanzar el controlador de referencia de Ryu.

```
ryu-manager ryu.app.simple_switch_13
```

- `simple_switch_13` actúa como un switch de **aprendizaje de MAC**.
- Se queda ejecutando; mostrará logs cada vez que lleguen paquetes nuevos.

3.2 Verificar conectividad con el controlador activo

Vuelve a la **Terminal 1** (Mininet) y ejecuta:

Listing 10: Prueba de conectividad total con controlador activo.

```
mininet> pingall
```

Resultado esperado

```
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

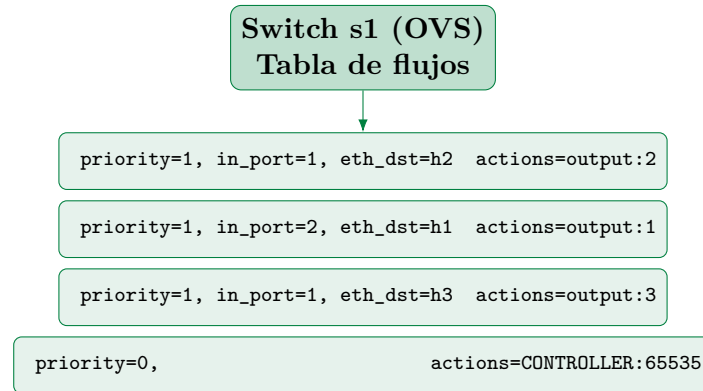
En la Terminal 2 verás mensajes como:

```
[INFO] packet in 1 xx:xx:xx:xx:xx:xx yy:yy:yy:yy:yy:yy 1
[INFO] packet in 1 ...
```

Concepto clave

El switch, al recibir un paquete *desconocido*, lo envía al controlador (**packet-in**). Ryu decide la acción y **instala una regla de flujo** para tráfico futuro, sin consultar de nuevo.

Paso 4: Inspeccionar flujos en el switch (Open vSwitch)



Flujos instalados por Ryu (aprendizaje MAC + tabla miss = enviar al controlador)

Figura 5: Representación de la tabla de flujos de s1 tras el pingall.

4.1 Ver flujos instalados en s1

Desde la consola de Mininet:

Listing 11: Volcado de flujos del switch s1.

```
mininet> sh ovs-ofctl dump-flows s1
```

Verás reglas con formato similar a:

```
cookie=0x0, duration=..., table=0, n_packets=...,
priority=1,in_port="s1-eth1",dl_dst=xx:xx:xx actions=
output:"s1-eth2"
```

Consejo

Punto clave para los estudiantes: estas reglas *no* fueron escritas manualmente. Las generó el controlador Ryu automáticamente al aprender las direcciones MAC de cada host.

Paso 5: Modificar el comportamiento de la red: firewall simple

5.1 Detener Mininet y Ryu

Listing 12: Detener sesiones antes de crear el firewall.

```
mininet> exit
```

En la Terminal 2, presiona **Ctrl + C** para detener ryu-manager.

5.2 Crear el directorio de trabajo y el script

Listing 13: Crear directorio sdn_lab y navegar a él.

```
cd ~
mkdir sdn_lab
cd sdn_lab
nano simple_firewall.py
```

5.3 Código del controlador simple_firewall.py

Pega el siguiente contenido en el editor:

Listing 14: simple_firewall.py — Controlador Ryu con política de bloqueo.

```
1 from ryu.base import app_manager
2 from ryu.controller import ofp_event
3 from ryu.controller.handler import (MAIN_DISPATCHER,
4                                     CONFIG_DISPATCHER,
5                                     set_ev_cls)
6 from ryu.ofproto import ofproto_v1_3
7 from ryu.lib.packet import packet, ethernet, ipv4,
8     ether_types
9
10 class SimpleFirewall(app_manager.RyuApp):
11     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
12
13     # Tráfico bloqueado: h1 (10.0.0.1) -> h2 (10.0.0.2)
14     BLOCK_SRC_IP = "10.0.0.1"
15     BLOCK_DST_IP = "10.0.0.2"
16
17     def __init__(self, *args, **kwargs):
18         super(SimpleFirewall, self).__init__(*args, **
19                                             kwargs)
20         self.mac_to_port = {}
21
22     # -- Evento: el switch se conecta al controlador
23     -----
```

```

21 @set_ev_cls(ofp_event.EventOFPSwitchFeatures,
    CONFIG_DISPATCHER)
22 def switch_features_handler(self, ev):
23     datapath = ev.msg.datapath
24     ofproto = datapath.ofproto
25     parser = datapath.ofproto_parser
26     # Regla miss: enviar paquetes desconocidos al
        controlador
27     match = parser.OFPMatch()
28     actions = [parser.OFPActionOutput(ofproto.
        OFPP_CONTROLLER,
29                                     ofproto.
        OFPCML_NO_BUFFER
        )]
30     self.add_flow(datapath, 0, match, actions)
31
32 # -- Instalar un flujo en el switch
    -----
33 def add_flow(self, datapath, priority, match,
34             actions, buffer_id=None):
35     ofproto = datapath.ofproto
36     parser = datapath.ofproto_parser
37     inst = [parser.OFPInstructionActions(
38             ofproto.OFPIT_APPLY_ACTIONS,
39             actions)]
40     kw = dict(datapath=datapath, priority=priority,
41             match=match, instructions=inst)
42     if buffer_id:
43         kw['buffer_id'] = buffer_id
44     datapath.send_msg(parser.OFPFlowMod(**kw))
45
46 # -- Evento: llega un paquete nuevo (packet-in)
    -----
47 @set_ev_cls(ofp_event.EventOFPPacketIn,
48             MAIN_DISPATCHER)
49 def _packet_in_handler(self, ev):
50     msg = ev.msg
51     datapath = msg.datapath
52     ofproto = datapath.ofproto
53     parser = datapath.ofproto_parser
54     in_port = msg.match['in_port']
55
56     pkt = packet.Packet(msg.data)
57     eth = pkt.get_protocol(ethernet.ethernet)
58
59     if eth.ethertype == ether_types.ETH_TYPE_LLDP:
60         return # ignorar LLDP
61
62     dst, src = eth.dst, eth.src
63     dpid = datapath.id
64     self.mac_to_port.setdefault(dpid, {})

```

```

63         self.mac_to_port[dpid][src] = in_port      #
           aprendizaje MAC
64
65         # -- Politica de firewall (capa 3)
           -----
66         ip_pkt = pkt.get_protocol(ipv4.ipv4)
67         if ip_pkt:
68             if (ip_pkt.src == self.BLOCK_SRC_IP and
69                 ip_pkt.dst == self.BLOCK_DST_IP):
70                 self.logger.info("BLOQUEADO: %s -> %s",
71                                 ip_pkt.src, ip_pkt.dst)
72                 return      # descarta: no instala flujo ni
                           reenvia
73
74         # -- Switch de aprendizaje para el resto del
           trafico -----
75         if dst in self.mac_to_port[dpid]:
76             out_port = self.mac_to_port[dpid][dst]
77         else:
78             out_port = ofproto.OFPP_FLOOD
79
80         actions = [parser.OFPActionOutput(out_port)]
81
82         if out_port != ofproto.OFPP_FLOOD:
83             match = parser.OFPMatch(in_port=in_port,
84                                     eth_dst=dst, eth_src=
85                                     src)
86             if msg.buffer_id != ofproto.OFP_NO_BUFFER:
87                 self.add_flow(datapath, 1, match,
88                               actions, msg.buffer_id)
89             return
90         else:
91             self.add_flow(datapath, 1, match, actions)
92
93         data = (msg.data
94                 if msg.buffer_id == ofproto.OFP_NO_BUFFER
95                 else None)
96         out = parser.OFPPacketOut(
97             datapath=datapath, buffer_id=msg.buffer_id,
98             in_port=in_port, actions=actions, data=data)
           datapath.send_msg(out)

```

Guarda el archivo en nano: **Ctrl + O**, Enter, **Ctrl + X**.

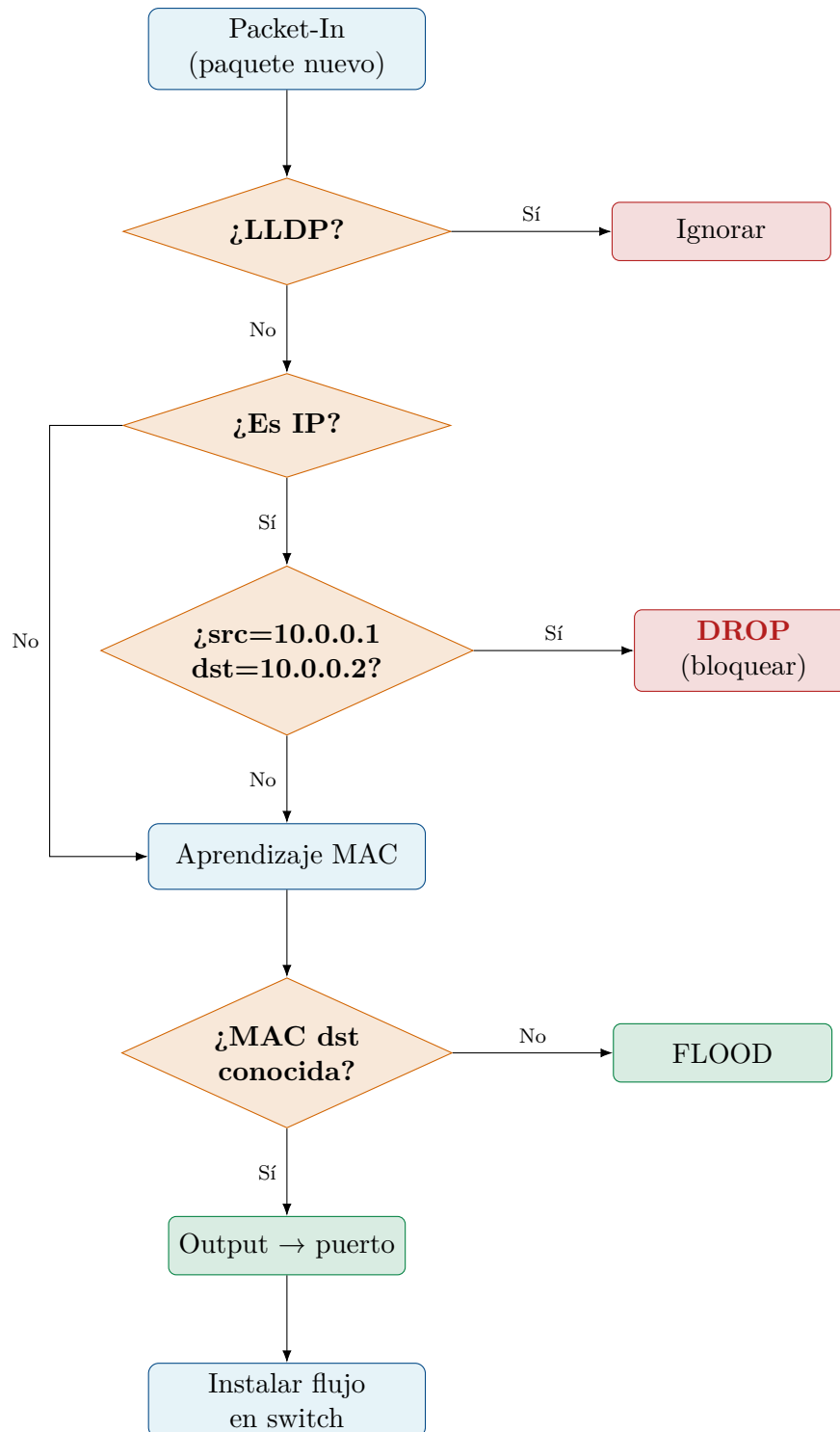


Figura 6: Diagrama de flujo del controlador SimpleFirewall.

Paso 6: Probar el controlador firewall en Mininet

6.1 Levantar Mininet (Terminal 1)

Listing 15: Levantar topología SDN de nuevo.

```
sudo mn --topo single,3 --controller=remote --switch ovsk
```

6.2 Ejecutar el firewall propio (Terminal 2)

Listing 16: Lanzar el controlador personalizado.

```
cd ~/sdn_lab
ryu-manager simple_firewall.py
```

Verás la línea:

```
[INFO] Loading SimpleFirewall
```

6.3 Verificar IPs de los hosts

Listing 17: Verificar IPs de h1, h2, h3.

```
mininet> h1 ifconfig
mininet> h2 ifconfig
mininet> h3 ifconfig
```

Host	IP	Interfaz
h1	10.0.0.1	h1-eth0
h2	10.0.0.2	h2-eth0
h3	10.0.0.3	h3-eth0

Cuadro 2: IPs asignadas por defecto en topología `single,3`.

6.4 Prueba 1: pingall

Listing 18: Prueba de conectividad general.

```
mininet> pingall
```


Resultado esperado (con firewall)

```

*** Ping: testing ping reachability
h1 -> X h3      <- bloqueado h1->h2
h2 -> h1 h3
h3 -> h1 h2
*** Results: 16% dropped (1/6 failed)

```

6.5 Pruebas específicas

Listing 19: Prueba $h1 \rightarrow h2$ (debe ser bloqueado).

```
mininet> h1 ping -c 3 10.0.0.2
```

Listing 20: Prueba $h2 \rightarrow h1$ (debe funcionar).

```
mininet> h2 ping -c 3 10.0.0.1
```

Listing 21: Prueba $h1 \rightarrow h3$ (debe funcionar).

```
mininet> h1 ping -c 3 10.0.0.3
```

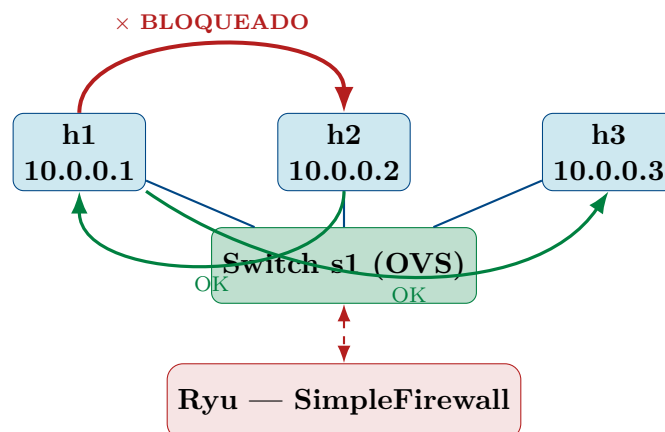


Figura 7: Resultado de las pruebas de conectividad con SimpleFirewall activo.

Observa en la **Terminal 2** (Ryu):

```

[INFO] BLOQUEADO: 10.0.0.1 -> 10.0.0.2
[INFO] BLOQUEADO: 10.0.0.1 -> 10.0.0.2
[INFO] BLOQUEADO: 10.0.0.1 -> 10.0.0.2

```

Paso 7: Variaciones propuestas y entregable

7.1 Variación 1: cambiar la regla de bloqueo

Edita `simple_firewall.py` y modifica las constantes:

Listing 22: Cambiar bloqueo a h3 → h1.

```
1 BLOCK_SRC_IP = "10.0.0.3"
2 BLOCK_DST_IP = "10.0.0.1"
```

Reinicia Ryu y repite los pings. Verifica el nuevo comportamiento.

7.2 Variación 2: bloquear todo el tráfico hacia un host

Modifica el bloque de firewall para bloquear *cualquier* origen hacia 10.0.0.2:

Listing 23: Bloquear todo el tráfico hacia h2.

```
1 if ip_pkt:
2     if ip_pkt.dst == "10.0.0.2":    # bloquear sin importar
        origen
3         self.logger.info("BLOQUEADO (destino h2): %s -> %s",
        " ",
4                             ip_pkt.src, ip_pkt.dst)
5     return
```

¿Qué resultado esperas? Prueba con `pingall` y con pings específicos.

7.3 Comparación Lab 1 vs. Lab 2

Aspecto	Lab 1 (VirtualBox + Ubuntu)	Lab 2 (Mininet + Ryu)
Topología	Redes internas + VM router	Hosts + switch OVS + Ryu
Configuración	CLI manual por VM	Script Python
Control	Distribuido (rutas estáticas)	Centralizado (Ryu)
Políticas	<code>ip route</code> en cada host	Reglas OpenFlow vía Ryu
Bloqueo	<code>iptables</code> por host	Lógica en el controlador
Escalabilidad	Limitada (por VM)	Alta (cientos de nodos virtuales)

Cuadro 3: Comparativa Lab 1 vs. Lab 2.

7.4 Entregable: reflexión escrita

Entregable / Actividad

Responde las siguientes preguntas de forma breve y clara (*máx. 1 párrafo cada una*):

P1. Rol del controlador SDN:

¿Cuál es el rol del controlador Ryu en esta práctica? ¿Qué pasaría si el controlador se detiene mientras los hosts están comunicándose?

P2. SDN vs. iptables:

¿En qué se diferencia bloquear tráfico con `simple_firewall.py` de hacerlo con `iptables` en cada host (como en el Lab 1)?

P3. Ventaja del control centralizado:

¿Qué ventaja observas en tener un único punto de control para toda la red? Menciona al menos un caso de uso real donde esto sería valioso.

P4. Extensión: (*opcional, puntos extras*)

Propón una política de red adicional que podrías implementar modificando `simple_firewall.py`. Descríbela en términos de código (qué campo verificarías, qué acción tomarías).

Resumen — Flujo completo del laboratorio

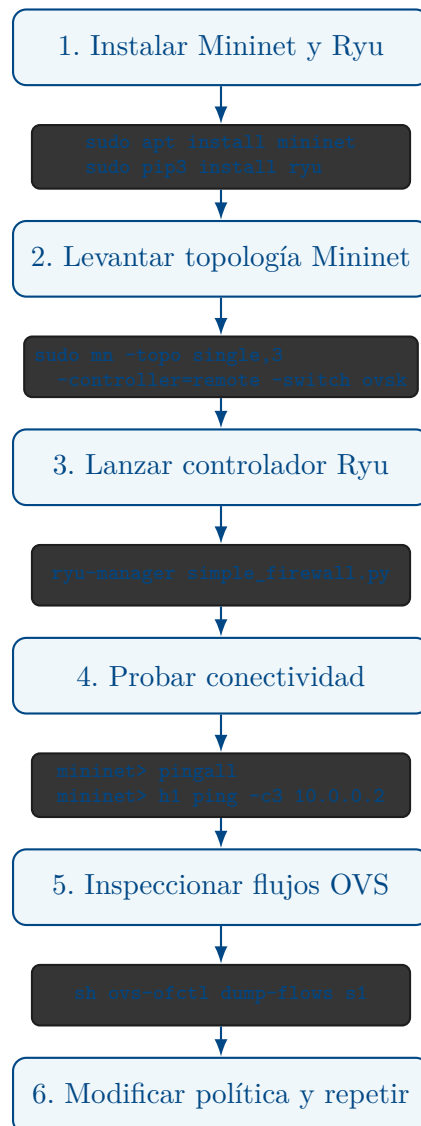


Figura 8: Resumen de los pasos del laboratorio con comandos clave.

Comando de limpieza (limpiar Mininet al finalizar)

```
sudo mn -c
```

Ejecuta este comando si Mininet reporta errores al reiniciar. Limpia todos los namespaces, interfaces virtuales y bridges de OVS creados anteriormente.

Bibliografía

- [1] **Doherty, J.** (2016). *SDN and NFV Simplified: A Visual Guide to Understanding Software Defined Networks and Network Function Virtualization*. Addison-Wesley Professional. 400 pp.
- [2] **Nadeau, T. D. & Gray, K.** (2013). *SDN: Software Defined Networks*. O'Reilly Media. 384 pp.
- [3] **Mininet Project.** (2024). *Mininet Walkthrough*. <http://mininet.org/walkthrough/>
- [4] **Ryu SDN Framework.** (2024). *Ryu Application API*. <https://ryu.readthedocs.io/>
- [5] **Open Networking Foundation.** (2015). *OpenFlow Switch Specification, Version 1.3.5*. <https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v3.5.pdf>